

Nama : Muhammad Eysar Assazily
NIM : F1D022070
Kelompok : 5

TUGAS PENDAHULUAN

MODUL IV

THREAD

1. Jelaskan apa yang dimaksud dengan proses thread! Kemudian dalam situasi apa saja thread biasanya digunakan?

Thread adalah unit eksekusi terkecil dalam sebuah proses yang memungkinkan program menjalankan tugas secara bersamaan (multitasking). Dalam sebuah proses, thread berbagi sumber daya seperti memori dan file, namun tetap memiliki counter, register, dan stack independen. Thread sering digunakan untuk meningkatkan performa aplikasi, terutama pada sistem multi-core, dengan memungkinkan tugas-tugas independen dijalankan secara paralel tanpa memblokir proses utama [1]. Situasi penggunaan thread biasanya meliputi aplikasi GUI, untuk menjaga antarmuka tetap responsif saat tugas berat berlangsung di latar belakang, aplikasi server untuk menangani banyak koneksi klien secara bersamaan, proses I/O agar operasi baca/tulis tidak memblokir eksekusi utama, simulasi dan pemrosesan paralel untuk memecah tugas besar menjadi sub-tugas yang dapat berjalan serentak, serta pengembangan game yang memanfaatkan thread untuk menjalankan logika game dan rendering grafis secara bersamaan [2].

2. Jelaskan perbedaan antara penggunaan single thread dan multithread! Sertakan kelebihan dan kekurangan masing-masing pendekatan!

=

Single thread adalah pendekatan di mana program hanya memiliki satu aliran eksekusi yang menangani semua tugas secara berurutan, sedangkan multithread menggunakan beberapa aliran eksekusi yang berjalan secara paralel untuk memungkinkan beberapa tugas dilakukan bersamaan dalam satu proses. Single thread lebih sederhana untuk diimplementasikan dan tidak memiliki risiko konflik data karena hanya ada satu aliran eksekusi, tetapi memiliki kelemahan berupa kecepatan eksekusi yang lambat untuk tugas berat dan kurang responsif, terutama saat menangani operasi yang membutuhkan waktu lama. Sebaliknya, multithread memungkinkan eksekusi paralel sehingga lebih cepat, responsif, dan optimal untuk tugas-tugas yang dapat dibagi ke beberapa thread, seperti aplikasi server atau GUI yang membutuhkan multitasking. Namun, pendekatan ini lebih kompleks karena memerlukan pengelolaan sinkronisasi antar thread, serta memiliki risiko masalah seperti deadlock atau race condition jika tidak dikelola dengan baik. Selain itu, multithread memanfaatkan lebih banyak sumber daya dibandingkan single thread [3].

3. Berikan contoh kode program sederhana yang menunjukkan penggunaan multithread di Java! Jelaskan fungsi setiap bagian kode tersebut!

```
class MyThread extends Thread {  
    private String threadName;  
    MyThread(String name) {  
        threadName = name;  
    }  
    public void run() {  
        for (int i=1; i<=5; i++) {  
            System.out.println(threadName + " - iterasi "  
                + i);  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                System.out.println(threadName + "  
                    Interrupted.");  
            }  
        }  
    }  
}
```

```

    }
}
public class MultithreadingExample {
    public static void main(String[] args) {
        MyThread t1 = new MyThread("Thread 1");
        MyThread t2 = new MyThread("Thread 2");
        t1.start();
        t2.start();
    }
}

```

Penjelasan kode:

- a. Membuat kelas MyThread
Kelas ini adalah subclass dari Thread. Dengan mewarisi kelas Thread, dapat dibuat logika untuk menjalankan tugas tertentu pada sebuah thread.
- b. Constructor untuk Thread
Constructor digunakan untuk memberi nama pada setiap thread sehingga memudahkan identifikasi saat program dijalankan.
- c. Override metode run()
Metode run() adalah tempat di mana logika eksekusi thread didefinisikan. Dalam contoh ini, setiap thread mencetak pesan 5 kali dengan jeda 500 ms antar iterasi menggunakan Thread.sleep(500).
- d. Menangani exception
Metode sleep() dapat memunculkan InterruptedException, sehingga perlu ditangani dengan blok try-catch.
- e. Kelas utama (MultithreadingExample)
Di dalam metode main(), dua objek MyThread dibuat dengan nama Thread 1 dan Thread 2. Metode start() dipanggil untuk memulai masing-masing thread. Metode ini secara otomatis memanggil run() pada thread terkait.

4. Tuliskan langkah-langkah yang diperlukan untuk membuat koneksi ke database menggunakan JDBC! Jelaskan setiap langkah dengan singkat namun jelas!

- a. Mengimpor library JDBC
import java.sql.*;
Library ini menyediakan kelas dan antarmuka seperti Connection, Statement, dan ResultSet yang diperlukan untuk

berinteraksi dengan database [4].

b. Memload driver JDBC

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Perintah ini memuat driver database yang diperlukan (contohnya MySQL). Driver bertanggung jawab untuk menghubungkan aplikasi java dengan database tertentu [4].

c. Membuat koneksi ke database

```
Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/db_name", "username", "password");
```

Format URL-nya adalah `jdbc:mysql://[host]:[port]/[database_name]`, ditambah dengan pemberian username dan kata sandi dari databasenya. Objek Connection yang dihasilkan akan digunakan untuk interaksi dengan database [4].

d. Membuat statement

```
Statement statement = connection.createStatement();
```

Statement digunakan untuk perintah SQL sederhana.

Penggunaan Prepared Statement untuk menghindari SQL Injection dan mendukung parameter dinamis [4].

e. Mengeksekusi query

```
ResultSet resultSet = statement.executeQuery("SELECT * FROM users");
```

`executeQuery()` digunakan untuk perintah SELECT (mengembalikan data) dan `executeUpdate()` digunakan untuk perintah INSERT, UPDATE, atau DELETE (mengubah data) [4].

f. Memproses data hasil query

```
while (resultSet.next()) {  
    System.out.println("ID: " + resultSet.getInt("id"));  
    System.out.println("Name: " + resultSet.getString("name"));  
}
```

ResultSet digunakan untuk membaca data yang diambil dari database dengan metode seperti `getInt()` atau `getString()` [4].

g. Menutup resource

```
ResultSet.close();  
statement.close();
```

```
connection.close();
```

Menutup ResultSet, Statement, dan Connection dilakukan untuk menghindari kebocoran memori. Penting dilakukan dalam blok finally untuk memastikan penutupan tetap dilakukan meskipun terjadi kesalahan [4].

5. Buat source code java untuk mengimplementasikan koneksi database dengan JDBC yang memenuhi ketentuan berikut:
- a. Fungsi untuk menambahkan data (Create)
 - b. Fungsi untuk membaca data (Read)
 - c. Fungsi untuk memperbarui data (Update)
 - d. Fungsi untuk menghapus data (Delete)

```
import java.sql.*;

public class DatabaseCRUD {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/nama_database";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "password";
    private Connection connection;

    public DatabaseCRUD() {
        try {
            connection = DriverManager.getConnection(
                DB_URL, DB_USER, DB_PASSWORD);
            System.out.println("Koneksi ke database berhasil!");
        } catch (SQLException e) {
            System.out.println("Koneksi gagal: " + e.getMessage());
        }
    }

    public void readData(String name, int age) {
        String query = "SELECT * FROM users";
        try (Statement stmt = connection.createStatement();
            ResultSet resultSet = stmt.executeQuery(query)) {
            System.out.println("Data dalam tabel:");
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String name = resultSet.getString("name");
            }
        }
    }
}
```

```

        ("name");
        int age = resultSet.getInt("age");
        System.out.println("ID: " + id +
            ", Name: " + name + ", Age: " +
            age);
    }
} catch (SQLException e) {
    System.out.println("Error saat membaca
    data: " + e.getMessage());
}
}

public void addData(String name, int age) {
    String query = "INSERT INTO users (name,
    age) VALUES (?, ?)";
    try (PreparedStatement stmt = connection.pre-
    paredStatement(query)) {
        stmt.setString(1, name);
        stmt.setInt(2, age);
        stmt.executeUpdate();
        System.out.println("Data berhasil di-
        tambahkan.");
    } catch (SQLException e) {
        System.out.println("Error saat mena-
        mbahkan data: " + e.getMessage());
    }
}

public void updateData(int id, String newName, int
    newAge) {
    String query = "UPDATE users SET name = ?,
    age = ? WHERE id = ?";
    try (PreparedStatement stmt = connection.pre-
    pareStatement(query)) {
        stmt.setString(1, newName);
        stmt.setInt(2, newAge);
        stmt.setInt(3, id);
        int rowsUpdated = stmt.executeUpdate
        ();
    }
}

```

```

        if (rowsUpdated > 0) {
            System.out.println("Data berhasil
diperbarui.");
        } else {
            System.out.println("Data dengan
ID " + id + " tidak ditemukan.");
        }
    }
} catch (SQLException e) {
    System.out.println("Error saat mempe-
rbarui data: " + e.getMessage());
}
}

public void deleteData(int id) {
    String query = "DELETE FROM users WHERE
id = ?";
    try (PreparedStatement stmt = connection.pre-
pareStatement(query)) {
        stmt.setInt(1, id);
        int rowsDeleted = stmt.executeUpdate();
        if (rowsDeleted > 0) {
            System.out.println("Data berhasil
dihapus.");
        } else {
            System.out.println("Data dengan
ID " + id + " tidak ditemukan.");
        }
    }
} catch (SQLException e) {
    System.out.println("Error saat
menghapus data: " + e.getMessage());
}
}

public void closeConnection() {
    try {
        if (connection != null && !connection.
isClosed()) {
            connection.close();
            System.out.println("Koneksi
ditutup.");
        }
    }
} catch (SQLException e) {

```

```

        System.out.println("Error saat
        menutup koneksi: " + e.getMessage());
    }
}

public static void main(String[] args) {
    DatabaseCRUD db = new DatabaseCRUD();
    db.addData("Eysar", 20);
    db.readData();
    db.updateData(1, "Rasya", 21);
    db.deleteData(1);
    db.closeConnection();
}
}

```

6. Berikan minimal 3 ide contoh aplikasi yang mengimplemen-
tasikan penggunaan thread dan jelaskan secara singkat
letak implementasinya!

=
a. Aplikasi Chat Real-time

Dalam aplikasi chat real-time, thread digunakan untuk
menangani komunikasi klien-server secara paralel. Server
menggunakan thread untuk menerima koneksi dari banyak
klien, sementara klien menjalankan thread untuk menerima
pesan masuk tanpa mengganggu antarmuka pengguna [4].

b. Aplikasi Pemrosesan File Besar

Aplikasi ini menggunakan thread untuk mempercepat pem-
rosesan file besar dengan membagi tugas menjadi beberapa
bagian. Setiap bagian diproses oleh thread terpisah secara
paralel, kemudian hasilnya digabungkan [4].

c. Aplikasi Pemesanan Tiket Online

Thread memungkinkan aplikasi menangani banyak permin-
taan pemesanan tiket secara bersamaan. Dengan sinkro-
nisasi, setiap thread memproses stok tiket secara aman
untuk mencegah race condition dan over booking, sehing-
ga memastikan data konsisten meskipun ada ribuan
permintaan paralel [5].

DAFTAR PUSTAKA

- [1] Z. A. W. Sugandi, Y. A. Nugraha, S. N. Anam, dan I. Darmayanti, "Implementasi Konsep Pemrograman Berorientasi Objek dalam Aplikasi Pembukuan Keuangan Penjual Jus Buah Menggunakan Bahasa Pemrograman Java," *Jurnal Ilmiah ITCIDA*, vol. 8, no. 1, hal. 1-8, Juni 2022.
- [2] V. Siahaan dan R. H. Sianipar, *Pemrograman Java Mulai dari Nol Sampai Master*, vol. 1, Sparta Publisher, 2018.
- [3] M. Wali, dkk., *Pengantar 15 Bahasa Pemrograman Terbaik Di Masa Depan (Referensi & Coding Untuk Pemula)*, PT. Sonpedia Publishing Indonesia, 2023.
- [4] A. H. R. N. Rahayu dan B. S. Rijal, *Pemrograman Berorientasi Obyek SMK/MAK Kelas XII*, Gramedia Widiasarana Indonesia, 2021.
- [5] M. Aman, "Pengembangan Sistem Informasi Wedding Organizer Menggunakan Pendekatan Sistem Berorientasi Objek pada CV Pesta," *Jurnal danitra Informatika dan Sistem Informasi*, vol. 1, no. 1, hal. 47-60, 2021.